Algorithm & Program Development

COMP 2233: Data Structures (Week 2)

Instructor: Assoc. Prof. Dr. Ömer Çetin

The Master Blueprint

The Problem & The Process

Understanding the software development lifecycle

The Algorithm



Concepts, rules, and real-world logic

The Architecture

Stages of problem-solving and strategic design

Two Paths to a Solution

Path A: Trial & Error

Experimental and experiential approach.

Intuition-driven and highly unpredictable.

Path B: Algorithmic Development

Structured, step-by-step methodology.

Repeatable, mathematically derived, and predictable.

The Software Development Pipeline



1. Analysis

Determining requirements, resolving and framing the core problem

2. Design

Planning architecture, technology, and developing the algorithm

3. Development

Implementing design, writing code, creating databases

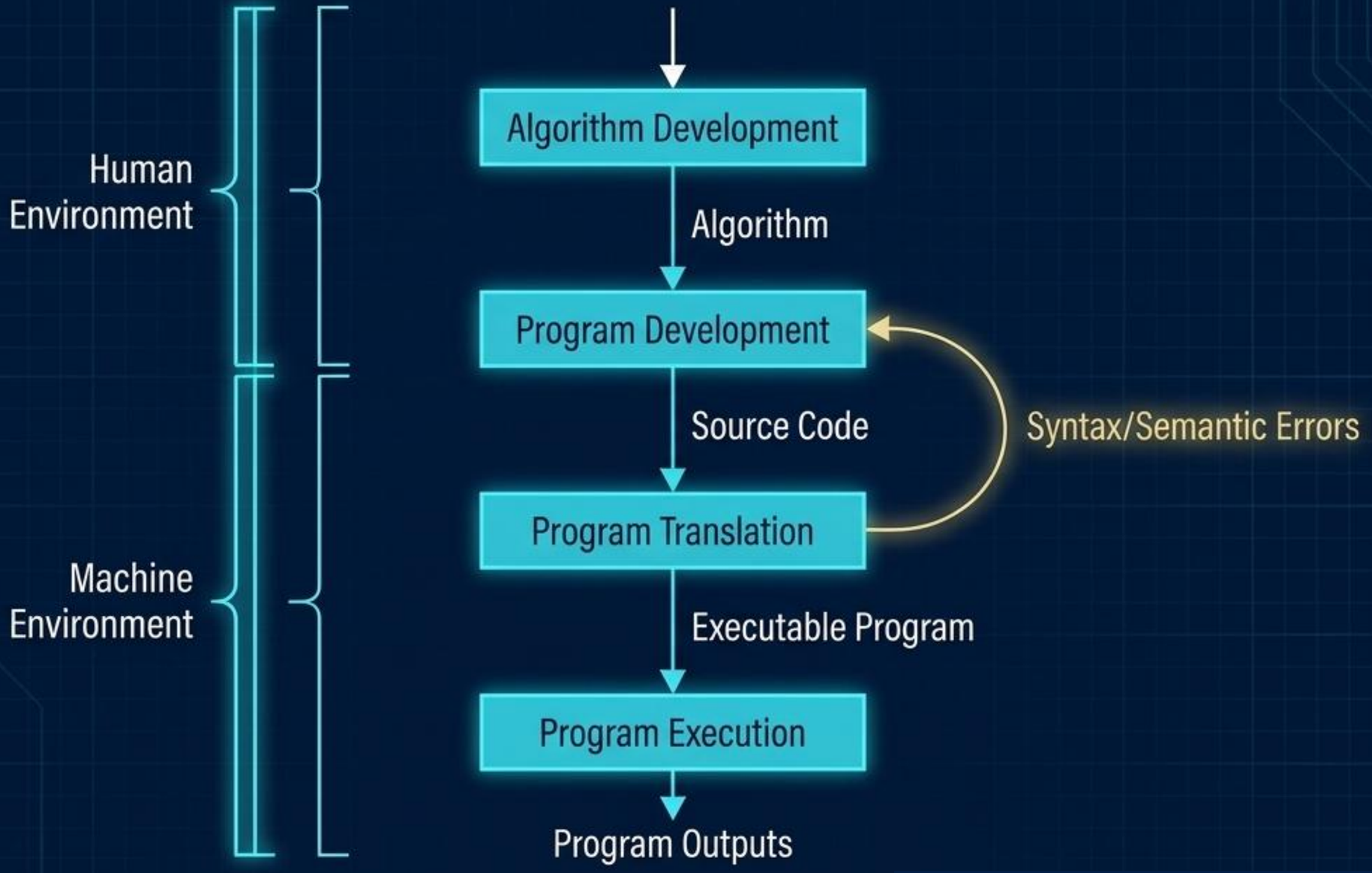
4. Debugging

Testing products, finding semantic/syntax errors, refining

5. Commissioning

Deploying software, training users

The Translation: From Human Thought to Hardware Execution



Translation Pipeline: Human Thought to Hardware Execution

1. HUMAN DOMAIN (Logic & Thought)

• Algorithm Development: Human drafts verbal steps, pseudocode or flowcharts.
• Program Development: Human writes source code (.cpp files).
• Susceptible to Semantic (Logic) Errors created by flawed human reasoning.

2. MACHINE DOMAIN (Compilation & Execution)

• Program Translation: Compiler (g++) checks Syntax Errors & builds binary.
• Program Execution: CPU & RAM execute machine code instructions.
• Program Outputs: Hardware returns calculated results to the user.

THE ERROR FEEDBACK LOOPS

A. Syntax Errors (Compiler Loop):
Violations of language grammar rules (e.g., missing semicolon, undeclared variable). Caught automatically during compilation. Developer fixes code syntax.

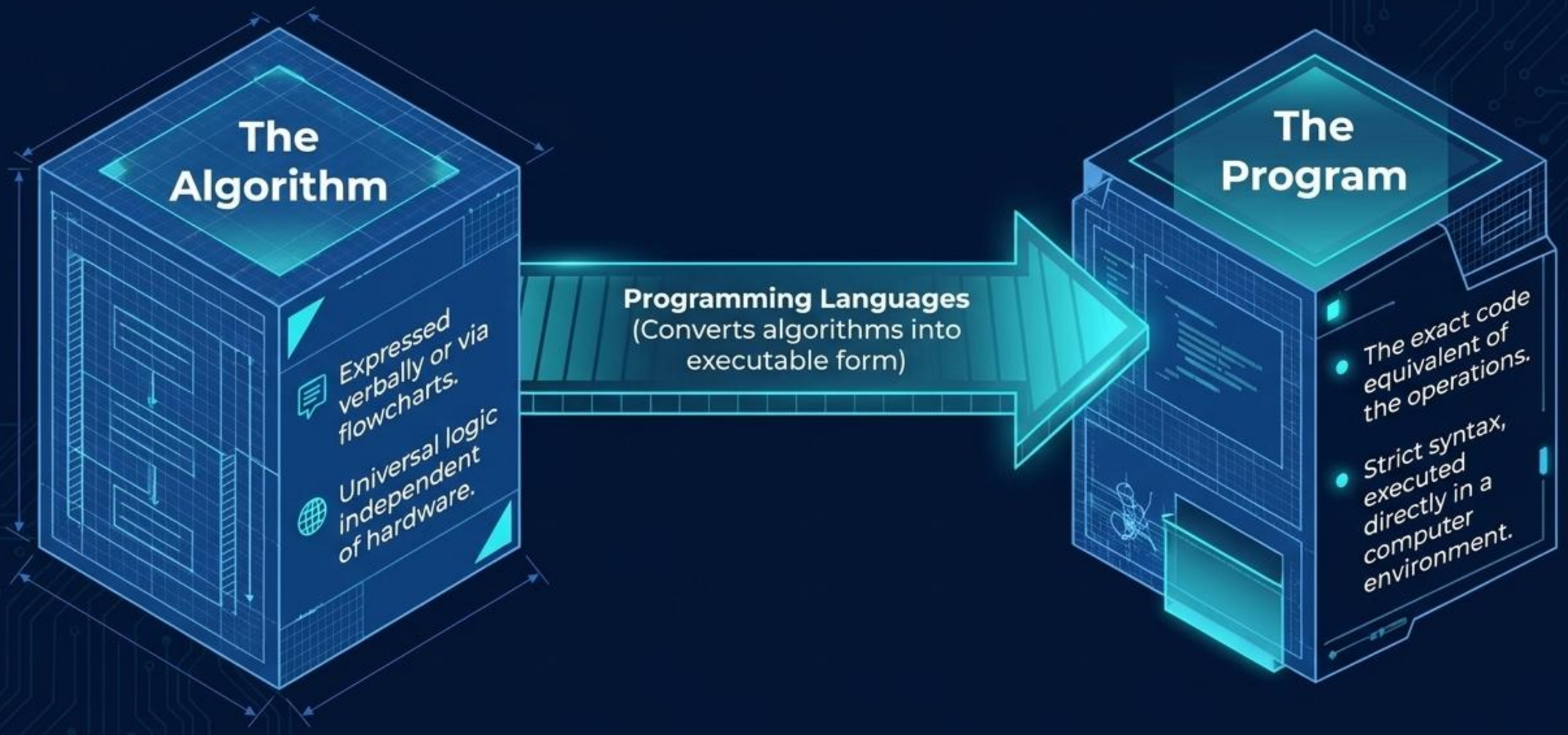
B. Semantic / Logic Errors (Runtime Loop):
Program compiles and runs cleanly, but produces INCORRECT results (e.g., calculating tax as addition instead of percentage). Caused by flawed algorithm design. Forces developer all the way back to Algorithm Development!

Defining the Algorithm

> “A verbal expression of how to carry out the logical solution of a problem step by step.”

Derived from the works of 9th-century Persian mathematician Muhammad ibn Musa al-Khwarizmi. Originally used in mathematics before entering computer science.

Algorithm vs. Program



The Three Pillars of an Algorithm



1. Precision

Steps must be unambigoous and have a single meaning. No vague expressions.



2. Strict Order

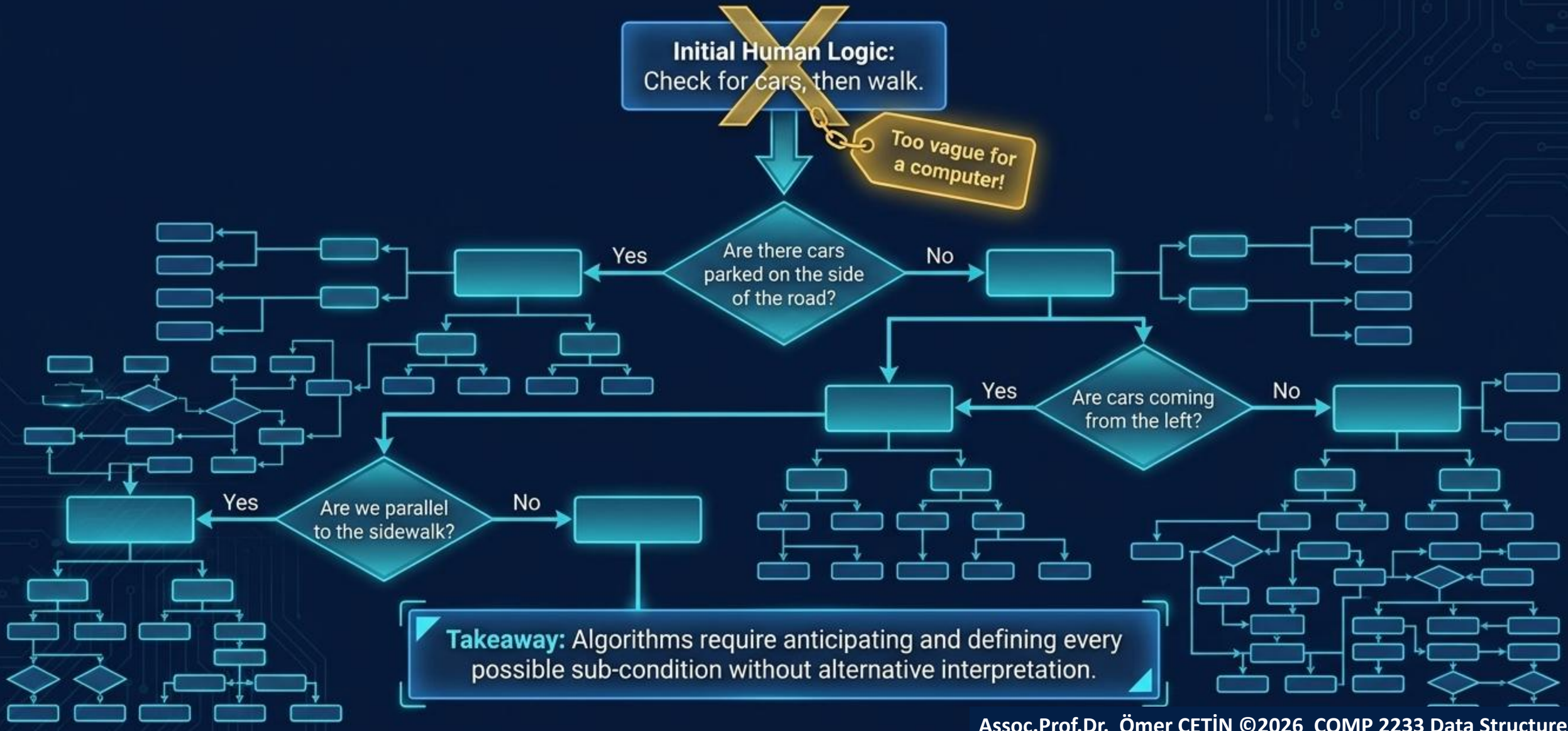
Requires a defined starting state. The sequence of steps must be strictly followed.



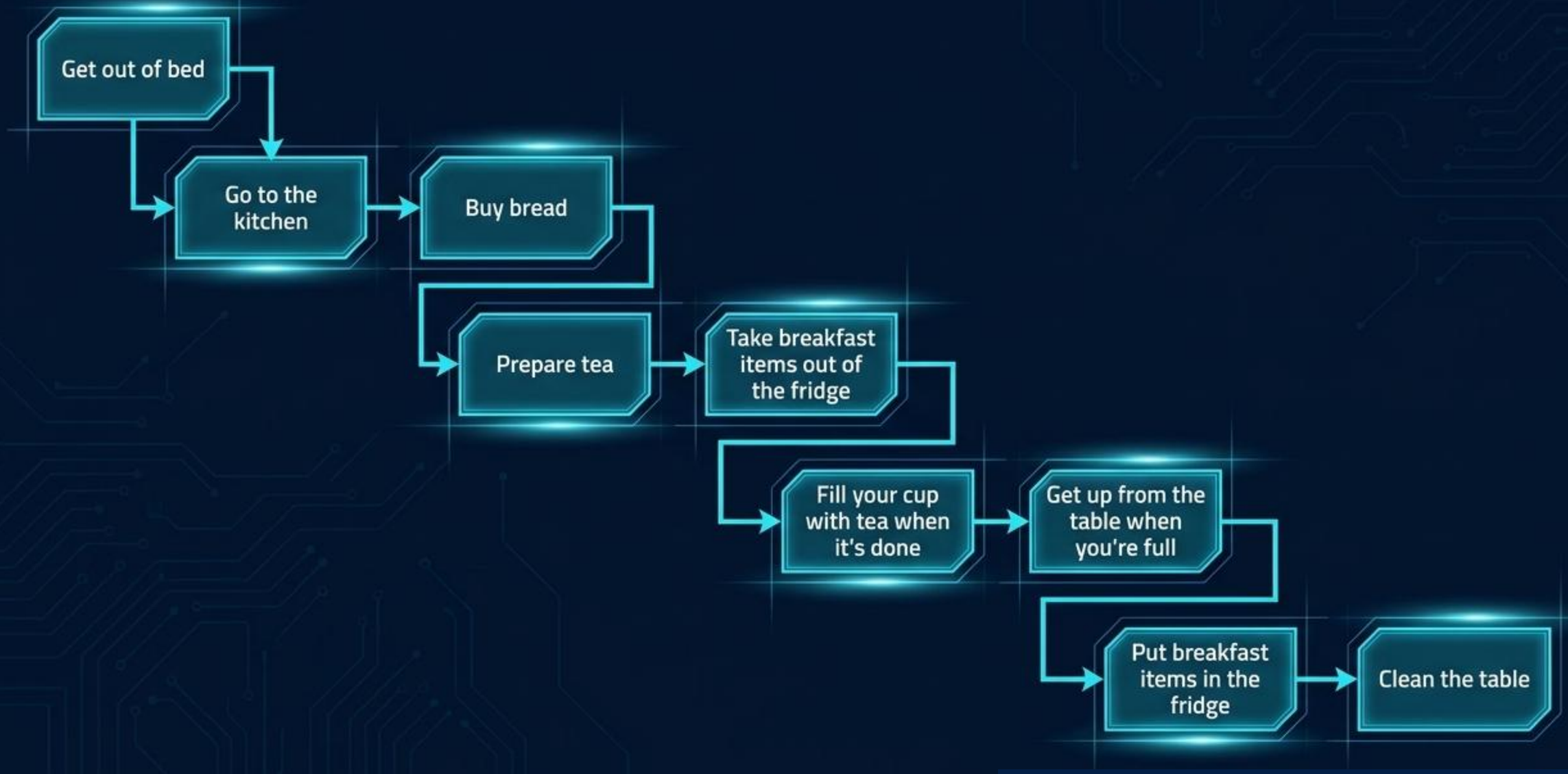
3. Finiteness

Must consist of a finite number of steps and end within a limited time period. Infinite loops fail this test.

Real-World Logic: Iterative Refinement



Sequential Execution: The Breakfast Algorithm



Everyday Algorithms: Imposing Algorithmic Order on Chaos

Example A: Breakfast Preparation (Linear Logic)

- Step 1: Get out of bed (Start).
- Step 2: Go to the kitchen.
- Step 3: Check if bread exists; if not, go to bakery.
- Step 4: Prepare tea and set items on fridge tray.
- Step 5: Eat breakfast (Core operation).
- Step 6: Clean table and put items away (Stop).
- Insight: Demonstrates sequential ordering & prerequisites.

Example B: Crossing the Road (Iterative Refinement)

- Step 1: Walk to pedestrian crossing (Start).
- Step 2: Look LEFT for incoming traffic.
- Step 3: Look RIGHT for incoming traffic.
- Step 4: Look LEFT again (Safety verification loop).
- Step 5: IF road is clear THEN cross ELSE wait 5s & repeat Step 2.
- Step 6: Reach opposite sidewalk (Stop).
- Insight: Demonstrates conditional branching & iterative loops.

Program vs. Algorithm: Abstract Logic vs. Executable Artifact

THE ALGORITHM (The Abstract Blueprint)

- A conceptual, step-by-step solution to a problem.
- Completely language-independent (no C++, Java, or Python syntax).
- Written in natural language, pseudocode, or flowcharts.
- Expressed on paper, whiteboard, or theoretical documentation.
- Evaluated by logical soundness, time/space complexity.
- Analogy: The architectural blueprint of a building.

THE PROGRAM (The Concrete Implementation)

- The concrete code equivalent transferred to a computer.
- Bound strictly to a target language syntax and compiler.
- Compiled into binary machine instructions or interpreted.
- Stored as executable files (.exe, binary ELF) in memory.
- Evaluated by actual CPU execution speed and RAM usage.
- Analogy: The physical skyscraper constructed from concrete & steel.

The Anatomy of Algorithmic Accuracy

Input
Values meeting the problem constraints.



Output
Desired results

The Engine
Executes step-by-step instructions in finite time.

Condition of Accuracy:
The algorithm MUST convert any valid input into the correct output.

Failure State:
If it yields wrong output or no output for even one valid input, the entire algorithm is considered wrong.

Assoc.Prof.Dr. Ömer ÇETİN ©2026 COMP 2233 Data Structures

Core Types of Algorithms



Search Algorithms



Sort Algorithms



Matrix or Vector Operation Algorithms

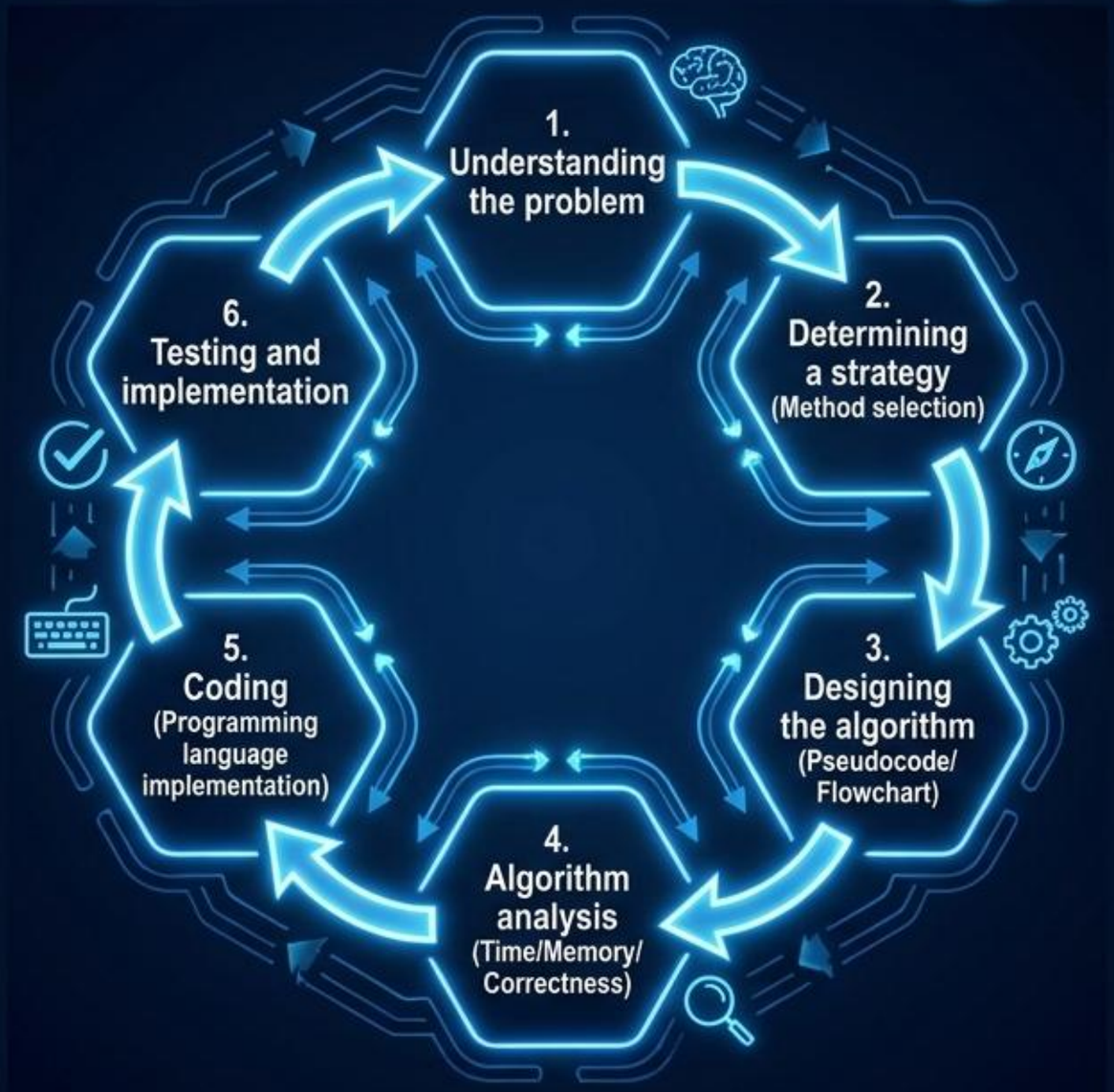


Graph Algorithms

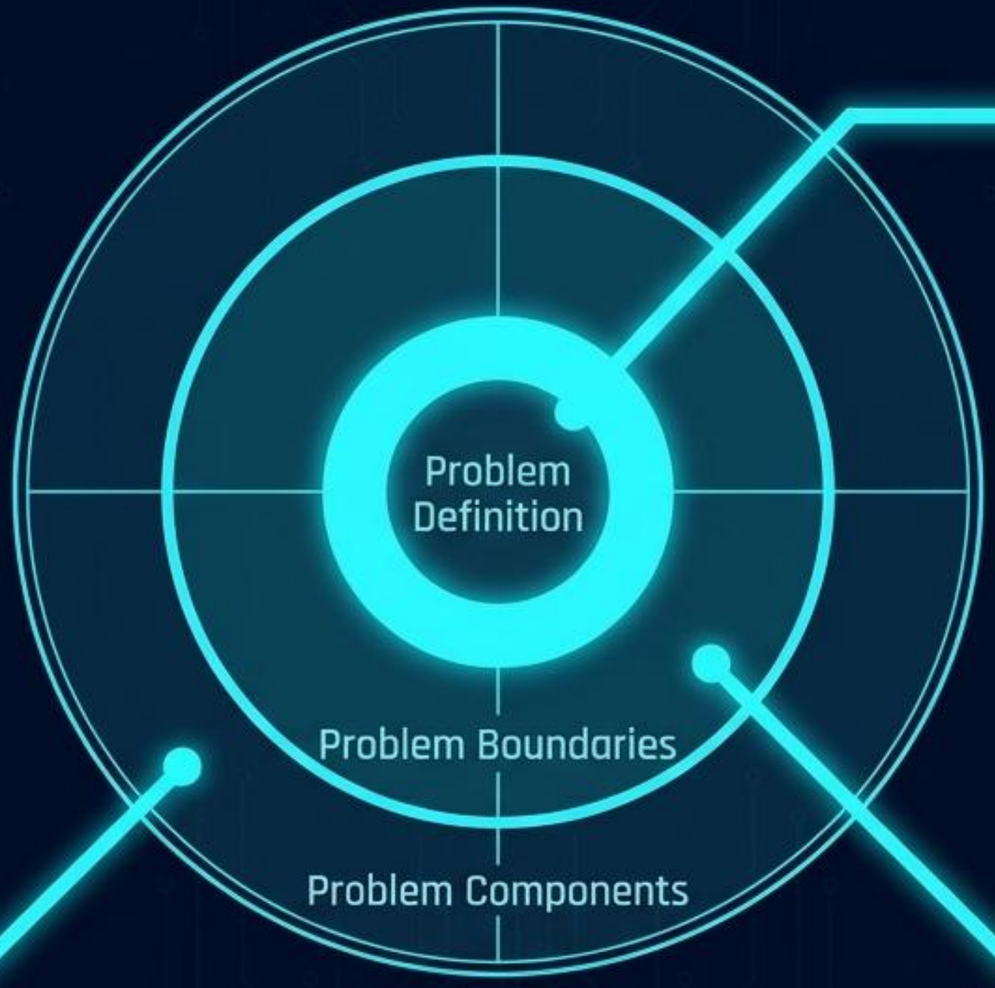


Mathematical Model Solvers

The 6 Problem-Solving Stages



Stage 1: Understanding the Problem



The problem must be clearly stated. The developer must ask examining questions to resolve ambiguities.

Dividing the core problem into manageable sub-problems to facilitate specific solution approaches.

Defining the scale and solution space. If unclear, design for the largest possible solution space.

Stage 2: Strategy Overview (The Arsenal)

Before writing the algorithm, a specific design method must be chosen to govern the logic.



Design Paradigms 1 & 2: Brute Force vs. Divide and Conquer

1. BRUTE FORCE METHOD (Exhaustive Search)

- The most straightforward, intuitive approach that comes to mind.
- Systematically checks every possible candidate solution.
- Generally produces naive, inefficient algorithms (O(N^2) or O(2^N)).
- However, highly effective for small inputs or specific problems.
- Classic Example: Matrix Multiplication (naive n^3 multiplications).
- Classic Example: Simple Linear Search through an unordered array.

2. DIVIDE AND CONQUER (Recursive Sub-Problems)

- Decomposes a complex problem into smaller independent sub-problems.
- Solves sub-problems recursively, then combines sub-solutions.
- Dramatically optimizes execution speed from O(N^2) down to O(N log N).
- Highly suited for parallel processing and recursive data structures.
- Classic Example: Merge Sort (split array, sort halves, merge).
- Classic Example: Quick Sort (partition around pivot & recursively sort).

Design Paradigm 3: Reduce and Manage (Decrease & Conquer)

Constant Reduction (Decrease by 1)

- Reduces the problem instance size by a fixed constant (usually 1).
- At each step, 1 element is processed/sorted and removed from scope.
- The remaining (N-1) problem instance is solved iteratively.
- Classic Application: Insertion Sort.
- In Insertion Sort, we take 1 unsorted element and insert it into an already sorted subarray of size (N-1).
- Time Complexity: O(N^2) worst-case, O(N) best-case.

Constant Factor Reduction (Decrease by Half)

- Reduces the problem instance size by a constant factor (usually 1/2).
- At each step, half of the search space is discarded entirely.
- Dramatically accelerates search queries in structured data.
- Classic Application: Binary Search.
- In Binary Search on a sorted array, comparing with the middle element eliminates 50% of remaining elements instantly.
- Time Complexity: O(log N) logarithmic time!

Design Paradigms 4 & 5: Transform vs. Space-Time Trade-off

4. TRANSFORM AND MANAGE

- Transforms the original problem into an alternative domain that is easier to solve.
- Stage 1: Instance transformation into a helper structure.
- Stage 2: Solve the problem in the transformed domain.
- Classic Example: Heap Sort (transforms array into a Binary Max-Heap tree structure before sorting).
- Classic Example: AVL Trees (transforms unbalanced BST into balanced tree via rotations).

5. MEMORY AND TIME TRADE-OFF

- Sacrifices extra memory (RAM) space to achieve dramatic gains in processing speed.
- Pre-allocates lookup tables or auxiliary arrays.
- Converts expensive runtime computation into fast memory retrieval.
- Classic Example: Hash Tables (direct O(1) key access via array indices).
- Classic Example: Counting Sort (uses frequency array to achieve O(N) sorting time).

Design Paradigms 6, 7 & 8: DP, Greedy & Iterative Optimization

6. Dynamic Programming

Memoization & Sub-Problems

- Solves complex optimization problems with overlapping sub-problems.
- Stores intermediate sub-results in a memory table (memoization).
- Prevents redundant re-calculation of identical sub-cases.
- Example: Longest Common Subsequence (LCS), Shortest Path (Floyd-Warshall).

7. Greedy Approach

Locally Optimal Choices

- At each execution step, makes the immediate best local choice.
- Hopes local decisions lead to global optimum.
- Fast & simple, but does not always guarantee globally optimal result.
- Example: Fractional Knapsack Problem, Prim's Minimum Spanning Tree.

8. Iterative Optimization

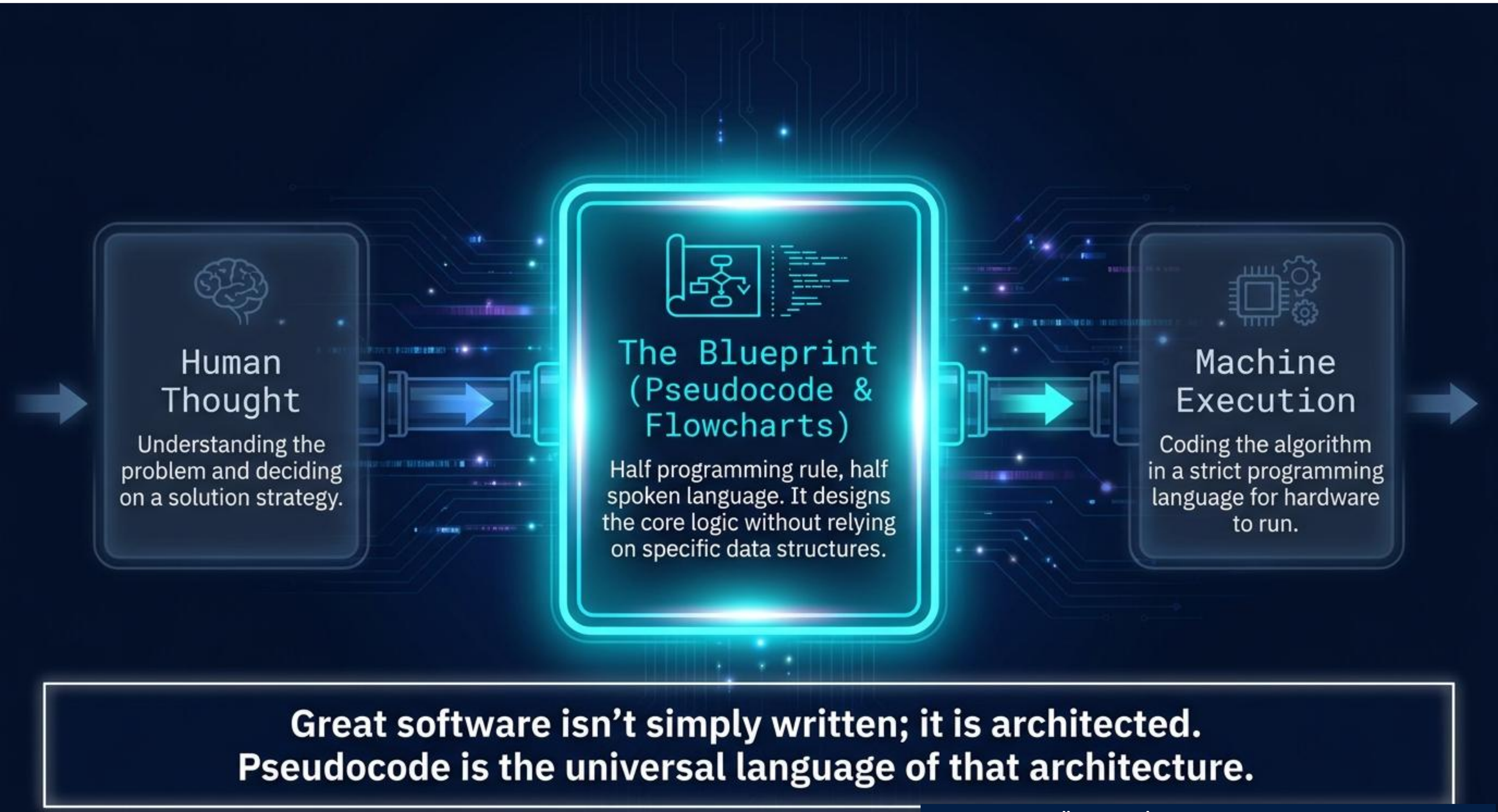
Gradient Improvement

- Starts with an initial feasible solution.
- Repeatedly improves the solution value at each iteration step.
- Continues until no further mathematical improvement is possible.
- Example: Simplex Algorithm for Linear Programming.

COMP 2233: DATA STRUCTURES

Stage 3: Designing an Algorithm

Pseudocode & Flowcharts


Human Thought
Understanding the problem and deciding on a solution strategy.
The Blueprint (Pseudocode & Flowcharts)
Half programming rule, half spoken language. It designs the core logic without relying on specific data structures.
Machine Execution
Coding the algorithm in a strict programming language for hardware to run.
Great software isn’t simply written; it is architected.
Pseudocode is the universal language of that architecture.

Stage 3: Algorithm Design — Pseudocode vs. Real Code

PSEUDOCODE (The Conceptual Blueprint)

- Half spoken language, half programming construct.
- Informal and completely language-agnostic.
- Independent of specific data structure syntax details.
- Designed for HUMANS to easily understand and review logic.
- Focuses purely on algorithmic correctness & step order.
- Example: 'IF list is empty THEN return NULL'.

REAL CODE (The Concrete Implementation)

- Strict programming language syntax (C++, Java, Python).
- Bound directly to concrete memory data structures.
- Requires exact memory allocation, pointers & variable types.
- Designed for COMPILERS to translate into machine instructions.
- Focuses on execution efficiency & syntax compliance.
- Example: 'if (head == nullptr) return nullptr;' in C++.

Pseudocode (The Blueprint)	Real Code (The Implementation)
Conceptual Blueprint	Concrete Implementation
Half-spoken & Flexible	Strict Programming Rules
Universal & Independent	Tied to Specific Data Structures
Designed for Humans to understand	Designed for Machines to compile

The Architect's Notation: Basic Operators

Assignment

:= <-

c <- 2πr

Mathematics

Arithmetic

+ - × / mod

Rounding

⌊ ⌋ (floor) ⌈ ⌉ (ceiling)

Set Operations

∑ (Summation)

∈ (Set membership)

Logic & Comparison

Comparison

= ≠ < > ≤ ≥

Logic

AND

OR

NOT

Core Keywords: The Verbs of Computing

I/O & Lifecycle Operations

START / END	Marks the boundaries of the algorithm.
INPUT	Captures data from a user via an input device.
READ / GET	Retrieves data from an existing file.
PRINT / DISPLAY / SHOW	Outputs results to a screen.

Processing Operations

COMPUTE / CALCULATE	Executes a mathematical expression.
SET / INIT	Assigns the initial value to a variable.
INCREMENT / BUMP	Increases a variable's value.
DECREMENT	Decreases a variable's value.

Structural Blueprints: Control Flow



```
if ... then ... else ...
```



```
while ... do
repeat ... until
for ... do
```

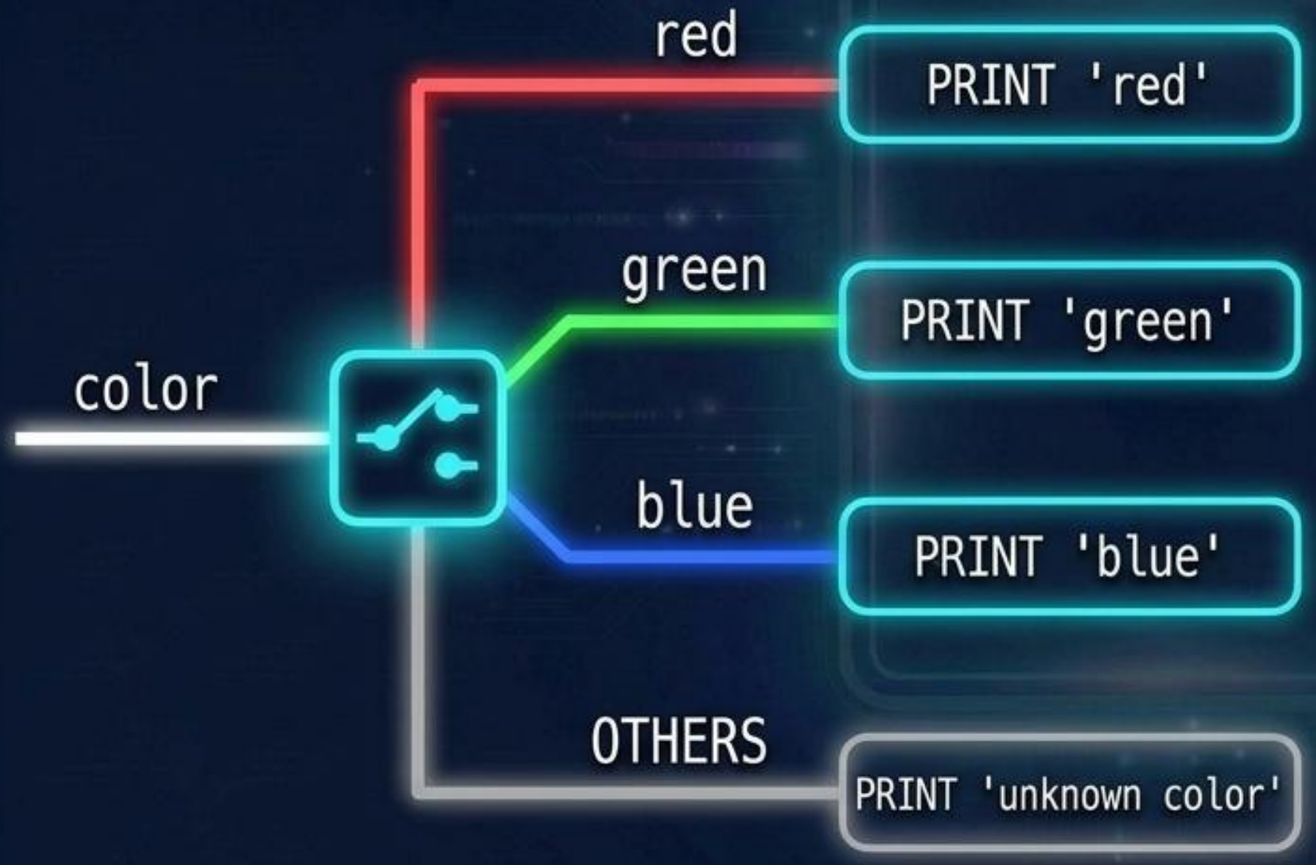


Array indices as `A[i]`
Method Calls as `Method_Name({parameters})` -> `return`

Logic in Action: Decision Making



Logic in Action: Multiple Branching



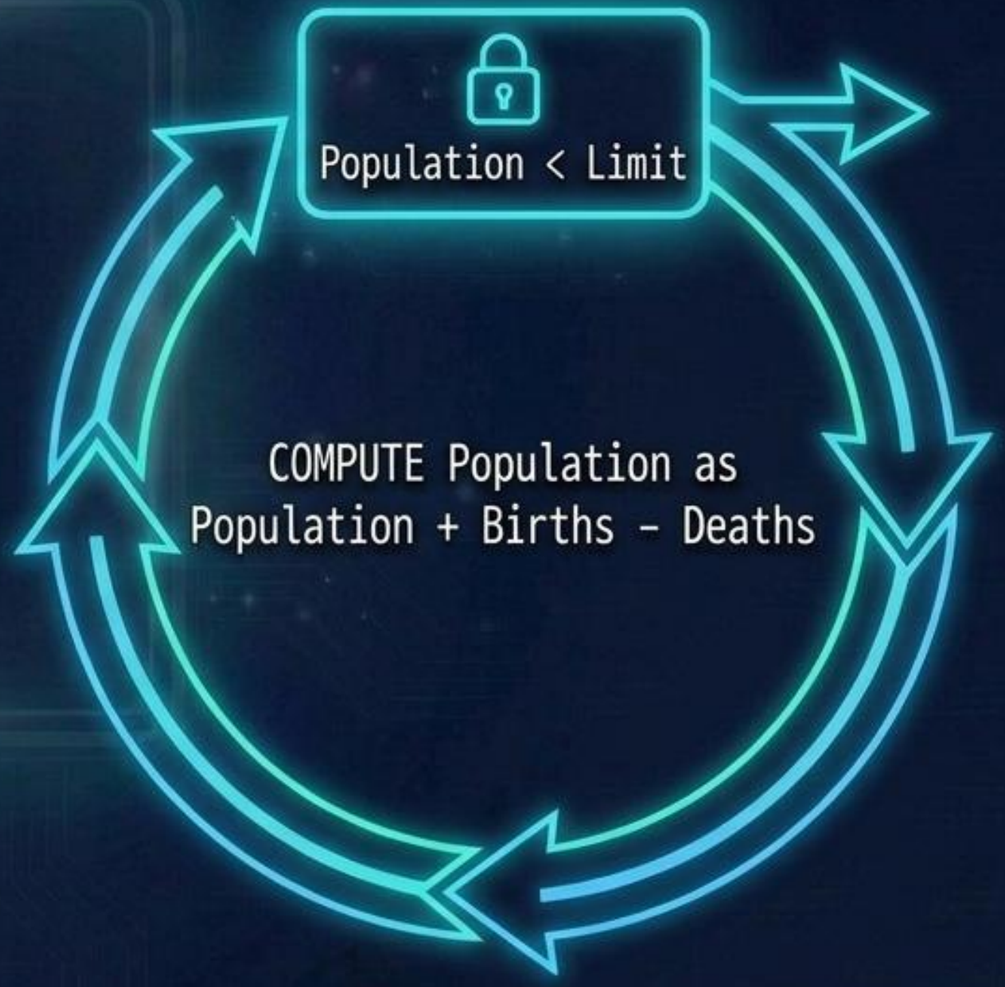

```
INPUT color
CASE color
   red: PRINT 'red'
   green: PRINT 'green'
   blue: PRINT 'blue'
   OTHERS: PRINT 'unknown color'
ENDCASE
```

Logic in Action: The Iteration (WHILE)

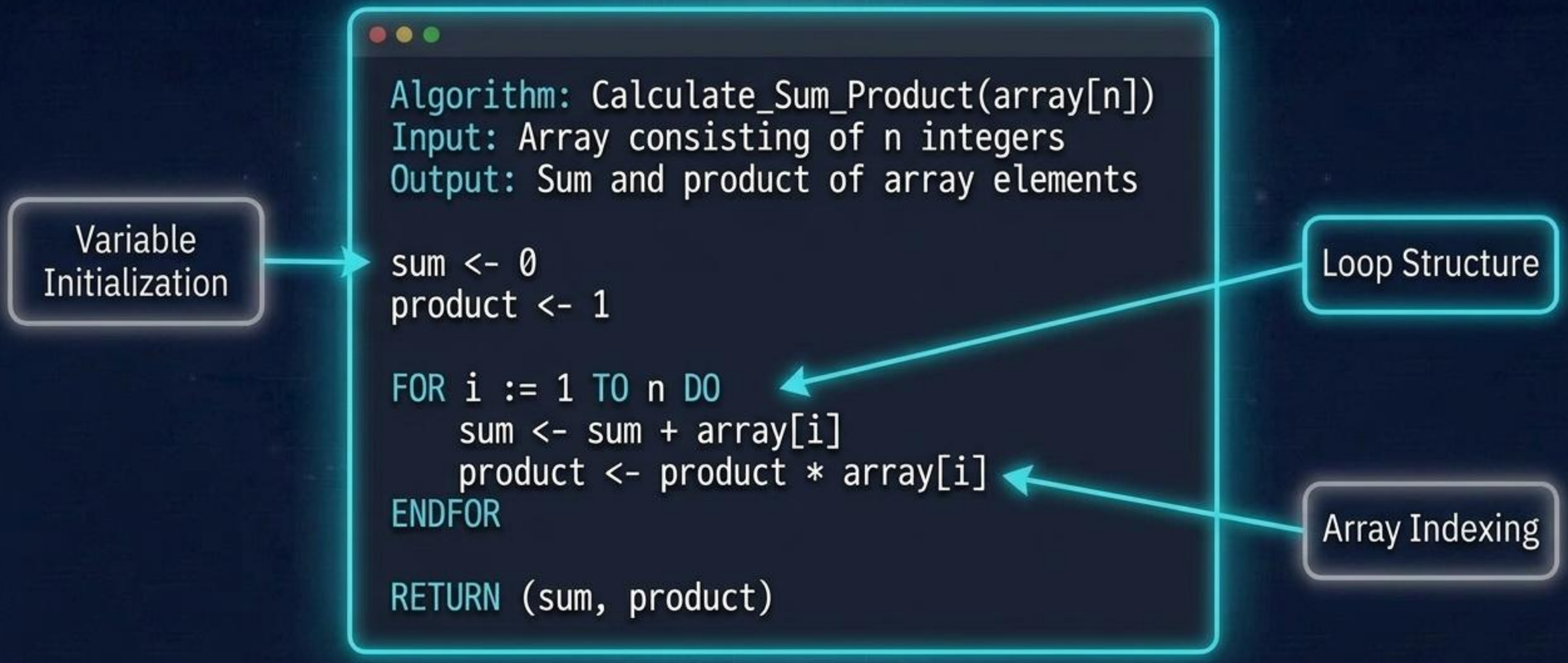
Precondition: SET variable X to 1

```
WHILE Population < Limit
  COMPUTE Population as Population + Births - Deaths
ENDWHILE
```

Key Insight: The loop only runs as long as the gatekeeper condition remains true.



Synthesis I: Array Calculation Algorithm



Synthesis II: From Pseudo to Real

The Skeleton (Pseudocode)

```
if (root points to empty tree)
    Add node to root;
else {
    if (added is less than root)
        branch to left subtree;
    else
        branch to right subtree;
}
```

The Muscles (C++ Code)

```
if (treeRoot == NULL)
    root = added;
else {
    if (added->info <= treeRoot->info)
        // branch left logic
    else
        // branch right logic
}
```

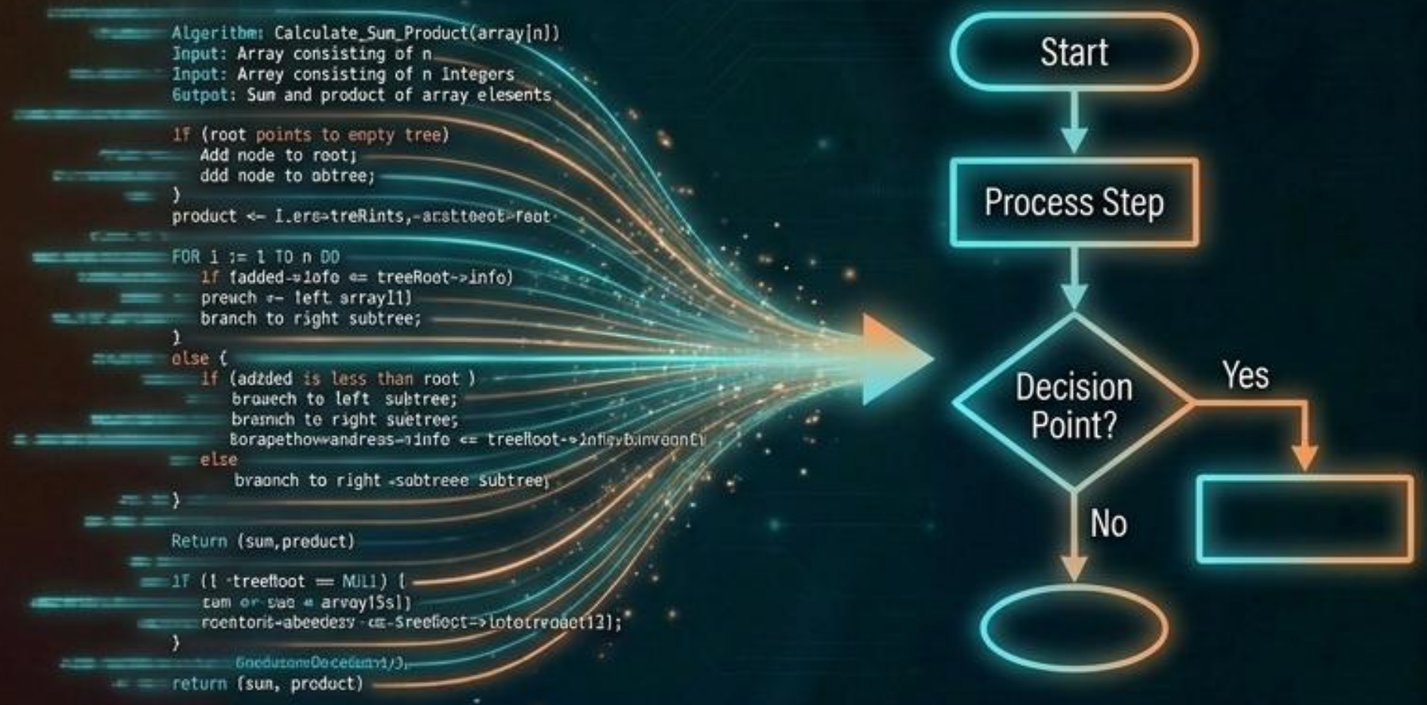
Pseudocode provides the exact skeleton;
Real Code adds the specific syntax muscles.

From Verbal to Visual.

Algorithms map the logic.

Pseudocode formats it.

Flowcharts visualize it.



A flowchart visually expresses arithmetic and logical steps using standardized geometric icons and directional arrows, leaving zero room for alternative interpretations.

The Visual Alphabet: Flowchart Symbols



Terminal: Indicates Start / End of the algorithm.



Input/Output: Indicates reading data (e.g., keyboard) or printing results.



Process: Indicates computing, calculating, or assigning variables.



Decision: Indicates branching logic (If/Else paths).



Flow: Indicates the strict direction of workflow.

Classification of Flowcharts: Linear, Logical & Circular

1. LINEAR FLOWCHARTS

Sequential Execution

- Operations execute sequentially from top to bottom.
- Contains zero decision branching or loop jumps.
- Straight execution path from START to STOP.
- Example: Calculating the sum of 2 numbers, computing circle area.

2. LOGICAL FLOWCHARTS

Decision Branching

- Incorporate logical decision nodes (Diamond symbols).
- Branches execution path based on True/False conditions.
- Simple calculation patterns, complex logical routing.
- Example: Checking if a number is positive/negative, comparing X and Y.

3. CIRCULAR FLOWCHARTS

Iterative Loops

- Incorporate loop pathways where steps repeat multiple times.
- Continues iteration until a counter or condition breaks the loop.
- Involves state initialization, increment, and loop checks.
- Example: Printing a string 5 times, summing array elements.

Example 1: Sequential Addition

Mapping steps to structural nodes.

Defining the logical algorithm before drawing the blueprint.

Algorithm Steps:

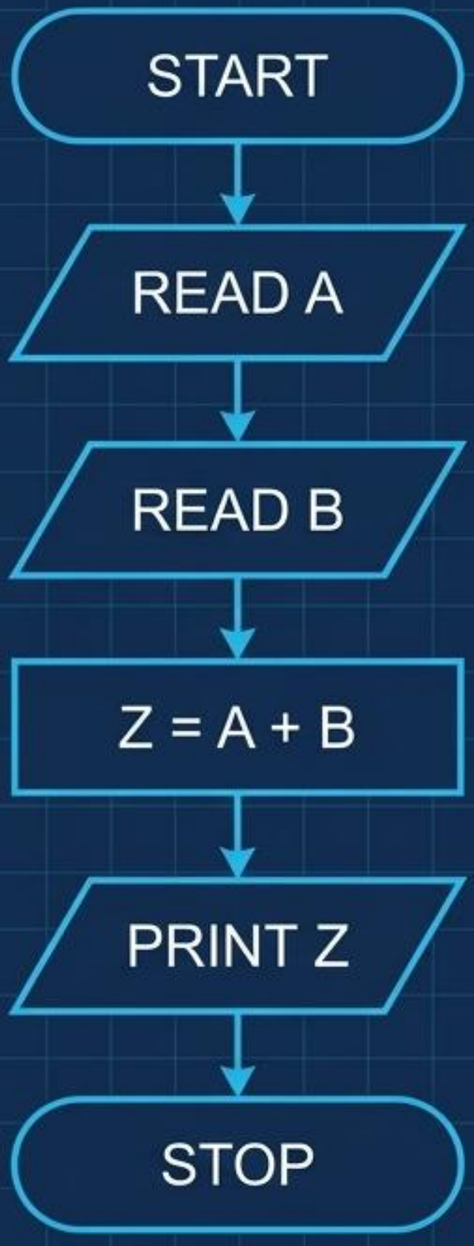
1. Start
2. Read First Number (A)
3. Read Second Number (B)
4. Calculate Total (Z = A + B)
5. Display Total (Z)
6. Stop

Variables Block:

A (First input)

B (Second input)

Z (Calculated Sum).

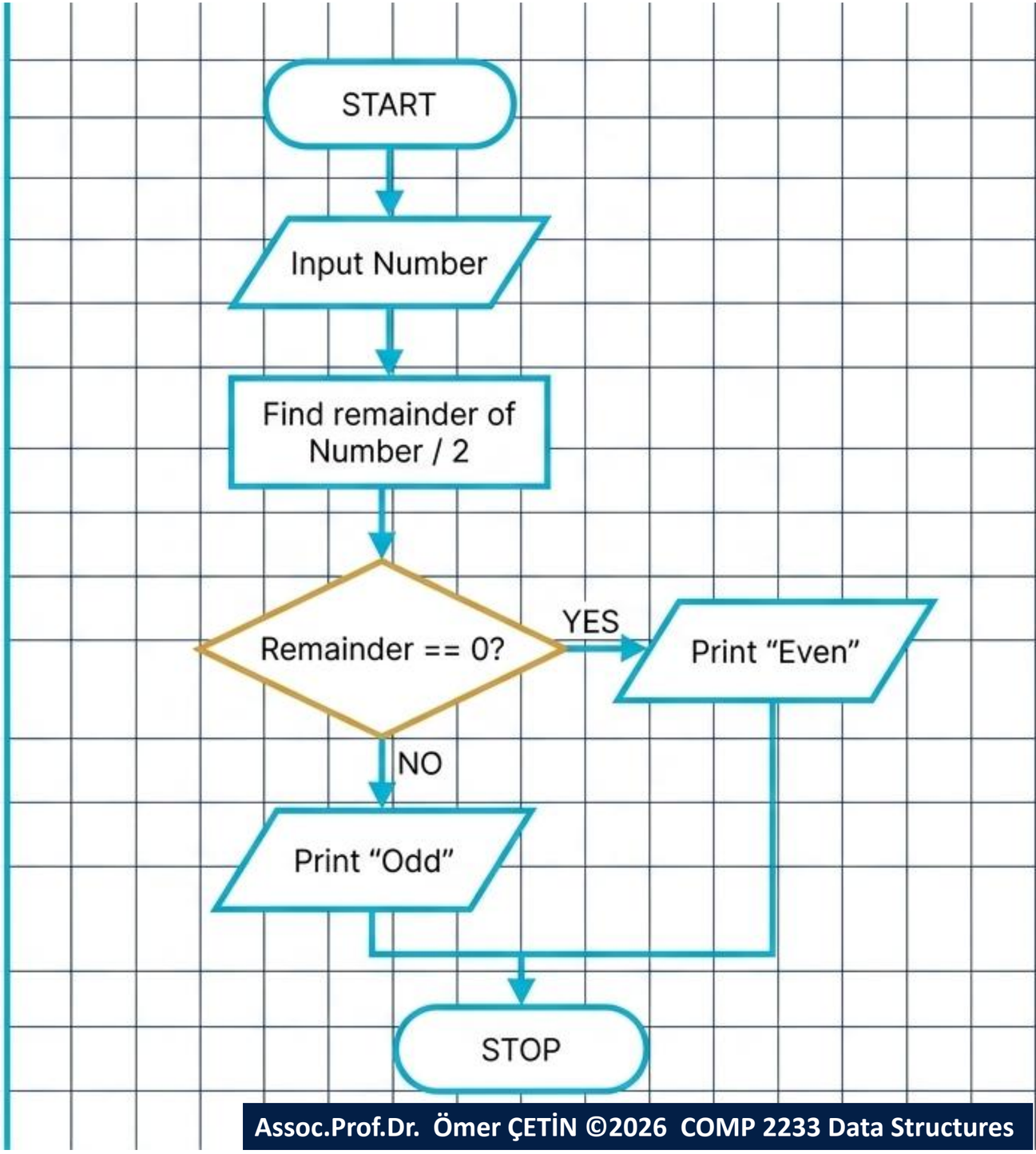


Example 2: Parity Checking (Odd or Even)

The diamond node forces a structural branch.

Algorithm Steps:

1. Start
2. Input a Number
3. Calculate remainder when divided by 2 (Modulo 2)
4. Is the remainder exactly 0?
5. If YES → Display “Even”
6. If NO → Display “Odd”
7. Stop

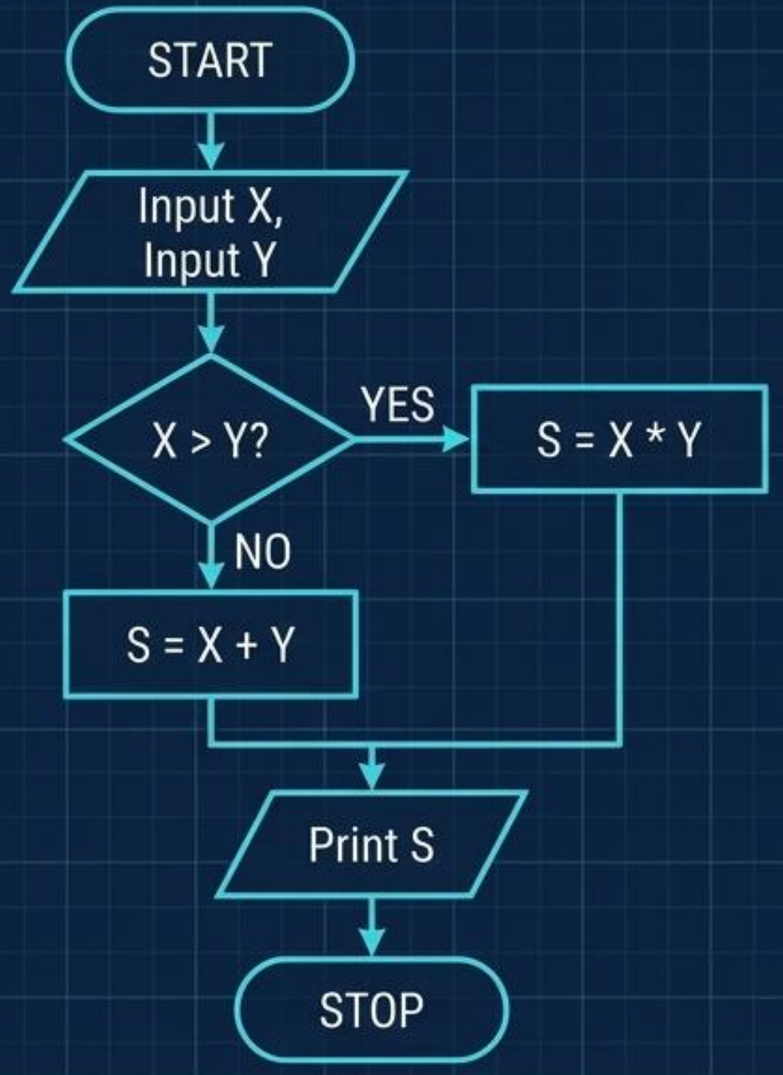


Example 3: Comparative Execution

Diverging processes, converging outputs.

Algorithm Steps:

1. Start
2. Read First Number (X)
3. Read Second Number (Y)
4. Condition: If X > Y, then Calculate S = X * Y
5. Condition: If X <= Y, then Calculate S = X + Y
6. Display Output (S)
7. Stop

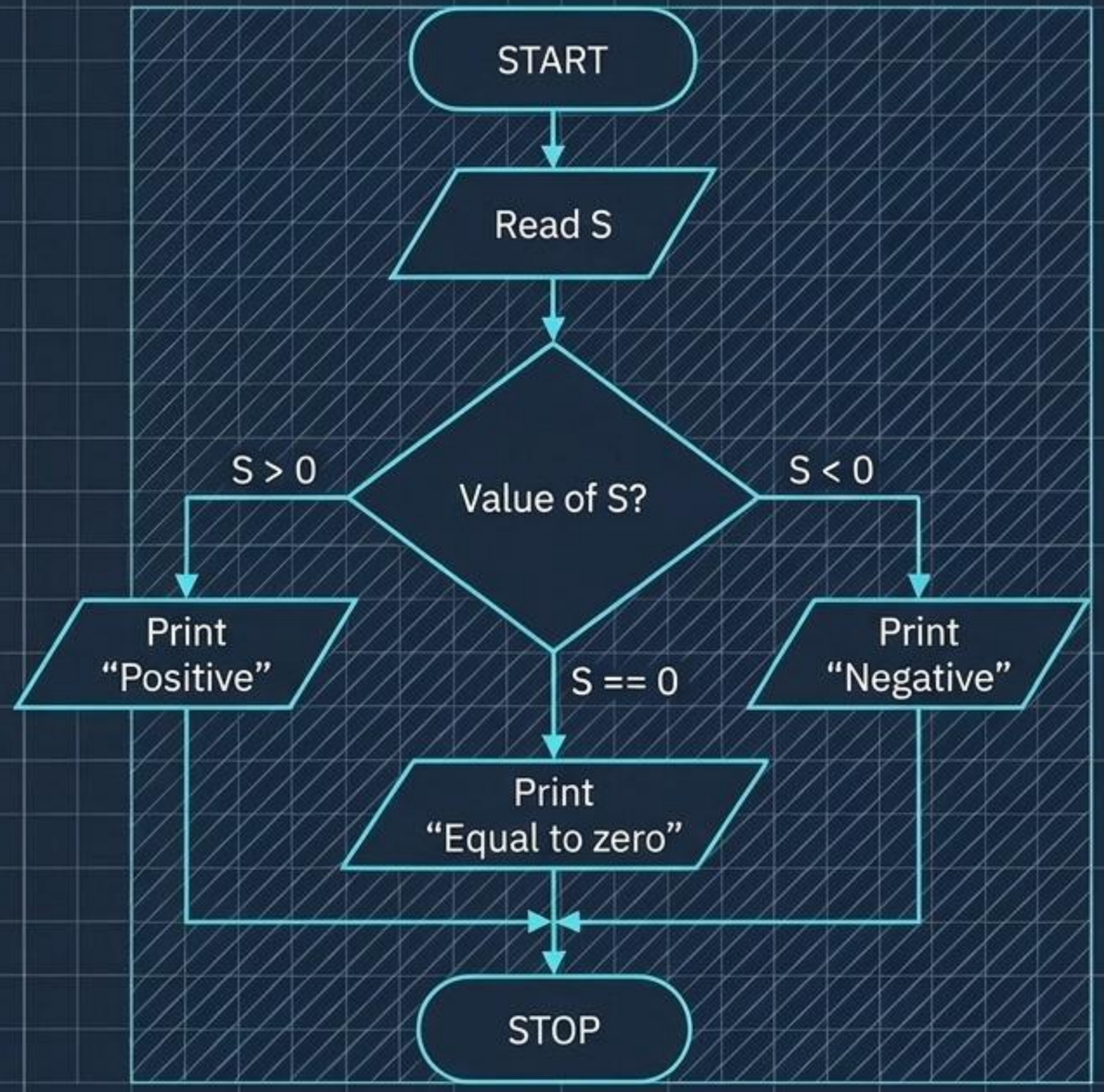


Example 4: Three-Way State Detection

Designing nested conditions (Positive, Negative, or Zero).

Algorithm Steps:

1. Start
2. Read Number (S)
3. If S > 0 → Print "Positive"
4. If S < 0 → Print "Negative"
5. If S == 0 → Print "Equal to zero"
6. Stop



Flowchart Taxonomy I: Linear Structures

Definition: In a linear structure, the execution of each operation is strictly determined by logical sequence. A subsequent operation cannot begin until the current one concludes.

Characteristics:

- Direct input → calculation → output workflows.
- No deviations, branches, or skipped steps.

Visual Archetype



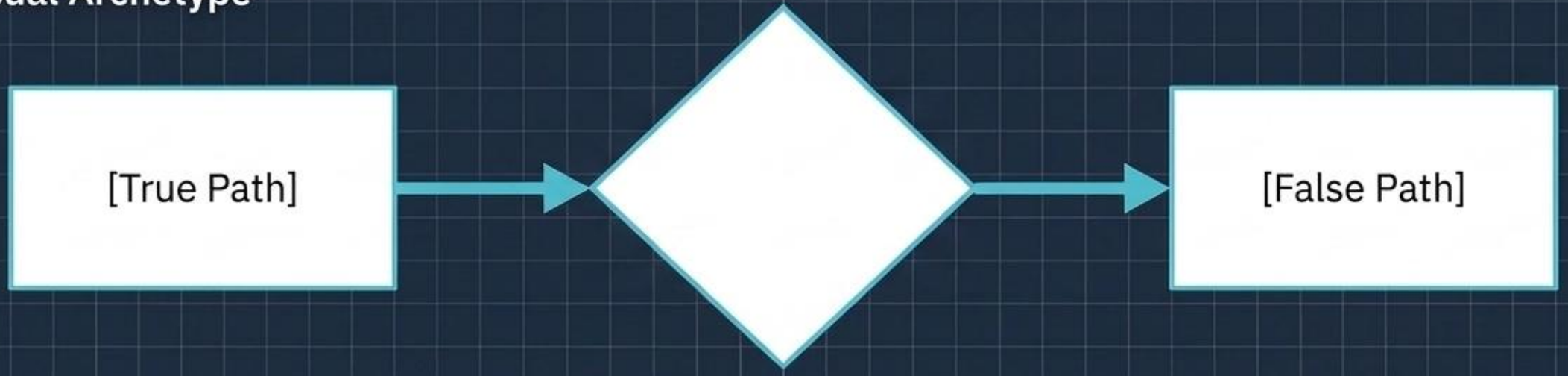
Flowchart Taxonomy II: Logical (Decision) Structures

Definition: Algorithms containing multiple sequential options. The structure utilizes conditions to determine which specific pathway will be executed.

Characteristics:

- Core to If...Then...Else programming logic.
- Calculation patterns are typically simple, but routing is complex.

Visual Archetype

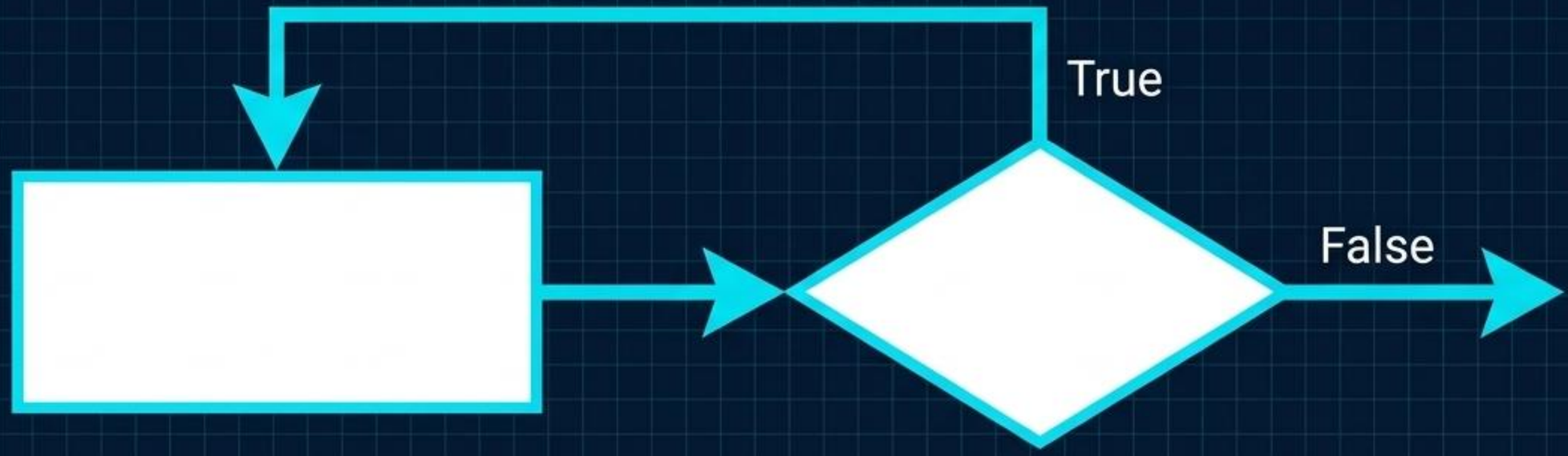


Flowchart Taxonomy III: Circular (Iterative) Structures

Definition: Flowcharts where a specific step or sequence is executed multiple times to solve the problem.

- **Characteristics:**
  - Creates programmatic loops (For, While).
  - Requires a known number of loops (start value, final value, increment) or a terminating condition.
  - Workflow includes initialization, calculation, and control feedback.

Visual Archetype



Stage 4: Algorithm Analysis (Correctness, Complexity & Optimality)

1. Proof of Correctness

Mathematical Verification

- Does the algorithm definitively solve the stated problem?
- Mathematically proves that valid inputs produce correct outputs.
- Verifies that all edge cases (empty lists, max bounds) are handled.
- Ensures termination under every execution path (Finiteness check).

2. Resource Complexity

Time and Space Analysis

- Processing Time Analysis: Asymptotic Big-O runtime complexity.
- Memory Space Analysis: RAM memory allocation footprint.
- Evaluates how execution scales as input size N approaches infinity.
- Trade-off evaluation: Sacrificing space for speed.

3. Optimality Verification

Is This the Best Solution?

- Evaluates whether a more efficient algorithm exists for this problem.
- Compares runtime against theoretical lower bounds $\Omega(N)$.
- Verifies if further optimization (DP, Greedy) can be applied.
- Ensures maximum software performance prior to writing C++ code.

Stage 5: Coding the Algorithm (Mapping Logic to C++ Syntax)

The Translation Mapping Phase

- Select the appropriate programming language (C++, Java, Python).
- Define variables & data types matching memory requirements.
- Map flowchart structures (Linear, Logical, Circular) to code constructs.
- Map Linear paths -> Sequential C++ statements.
- Map Decision diamonds -> C++ 'if-else' / 'switch-case' blocks.
- Map Circular loops -> C++ 'while', 'for', or 'do-while' loops.
- Implement precise Data Structures (Arrays, Pointers, Lists) to hold data.

Code Mapping Example (BST Insertion)

```
/* Pseudocode Insertion */
IF tree is empty THEN
    root := new_node
ELSE IF new_value < root.data THEN
    insert(root.left, new_value)
ELSE
    insert(root.right, new_value)

/* C++ Implementation */
if (root == nullptr) {
    root = newNode;
} else if (val < root->data) {
    insert(root->left, val);
} ...
```

Stage 6: Testing, Debugging & Execution Feedback Loops

1. Compilation & Syntax Verification

- Code is submitted to the C++ compiler (g++ -std=c++17).
- Compiler verifies strict programming language grammar rules.
- Catches typos, missing semicolons, undeclared variables & type mismatches.
- Syntax Errors prevent binary compilation completely.
- Developer fixes syntax bugs and re-compiles until 0 errors remain.

2. Runtime Execution & Semantic Debugging

- Executable binary is run on CPU with comprehensive test suites.
- Executes test cases including normal inputs, zero bounds & edge cases.
- Isolates Semantic / Logic Errors (incorrect output calculation).
- Isolates Runtime Crashes (segmentation faults, divide by zero, memory leaks).
- Performance profiling verifies if runtime matches Big-O analysis.

Lecture 2 Review, Homework Assignment & Week 3 Preview

HOMEWORK ASSIGNMENT 1: Circle Evaluator

- Goal: Reinforce Week 2 pseudocode design & flowcharting skills.
- Problem: Read circle radius R in centimeters from user (R > 0).
- Formulas: Perimeter P = 2 × π × R, Area A = π × R^2 (Assume π = 3.14159).
- Task 1: Write a complete structured Pseudocode algorithm.
- Task 2: Draw a formal ISO Flowchart showing all node symbols.
- Submission: Upload PDF/Word assignment to Moodle before Week 3.

WEEK 2 SUMMARY & WEEK 3 PREVIEW

- Week 2 Summary: Mastered SDLC pipeline, 6 Problem-Solving Stages, 8 Algorithm Design Paradigms, Pseudocode notation & Flowchart standards.
- Key Takeaway: Clean algorithm design reduces development bugs by 80%.
- Preview of Week 3: 'Arrays & Vectors (Single Dimension)'
- Week 3 Topics: Memory layout of arrays, contiguous allocation, C++ STL std::vector, dynamic sizing & element insertion/deletion algorithms.
- Preparation: Review C++ array declaration and pointer concepts!

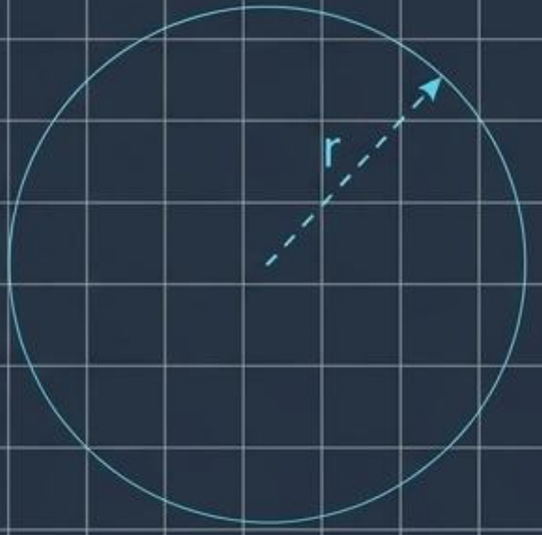
Weekly Assignment: Circle Geometry

Stage 1 & 2: Define the Problem

The Prompt:

- Design an algorithm and accompanying flowchart that calculates the area and perimeter of a circle.

- **Input Requirement:** The program must prompt the user to input the radius of the circle in centimeters (cm).



Assoc.Prof.Dr. Ömer ÇETİN ©2026 COMP 2233 Data Structures

Weekly Assignment: System Constraints

Stage 3: Logical Design Requirements

Output Conditions:

- Calculate Area (pi * r^2)
- Calculate Perimeter (2 * pi * r)
- Evaluate Perimeter Length:
  - Condition A: If Perimeter <= 100 cm → Print output as "narrow".
  - Condition B: If Perimeter > 100 cm → Print output as "wide".

Deliverable: Submit both the step-by-step text algorithm and the corresponding flowchart diagram.

Assoc.Prof.Dr. Ömer ÇETİN ©2026 COMP 2233 Data Structures

Looking Ahead: Week 3 Preview

Transitioning from standalone variables to structured data.

Upcoming Topics:

- Data Models and Memory Mapping
- Single Dimension Arrays
- Vectors and Continuous Memory

"Now that you can blueprint the logic, we will learn how to blueprint the data."